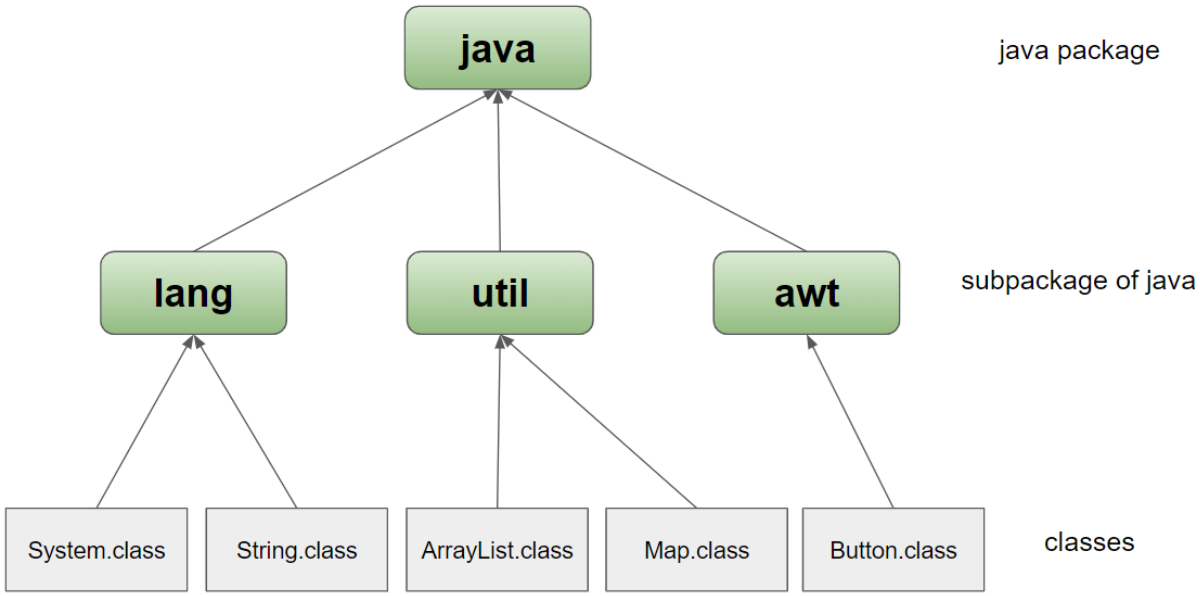
Laboratorul 1 - Java Basics

Programare procedurală
- instrucțiuni executate pas cu pas

Programare orientată obiect
- componente și interacțiunea dintre ele
- modelarea situațiilor din viața reală

Avantaje POO
- Readability
- Extensibility
- Testability
- Reusability
- Documentation

Organizarea unui proiect în pachete



Tipuri primitive
- boolean, char, float, int, long, etc
- există și wrappere pentru fiecare tip (Char, Integer, etc)

Clase
- tipuri de date definite de utilizator
- pot conține membri, proprietăți și metode
- obiecte = instanțieri ale clasei

ex: Integer x = new Integer(0)
- Declarare
- Instanțiere
- Atribuire

Câmpuri (membri)
- au tip primitiv sau tipul unei clase
- poate fi accesat folosind sintaxa className.memberName

Proprietăți
- câmp + getter + setter
- nu modificăm/accesăm direct starea

Metode (funcții membru)
- sintaxa className.methodName(param1, param2, ..., paramN)

Specificatori de acces
- private = acces doar în cadrul clasei curente
- default = acces doar în cadrul pachetului
- protected = acces doar în cadrul clasei curente + în cele care o moștenesc
- public = acces complet

Încapsulare
- acumularea metodelor și caracteristicilor comune în clase
- izolarea modului de implementare a unei clase

Observații
- câmpurile neinițializate explicit vor fi null/0/false
- numele fișierului = numele clasei publice din el

Ce se va afișa?

```
class Pair {
  int first;
  int second;
}

public class Main {
  public static void main(String[] args) {
    Pair firstPair = new Pair();
    firstPair.first = 1;

    Pair secondPair = new Pair();
    secondPair.first = 10;

    System.out.println(firstPair.first);
  }
}
```

- 1
- 10

Fiecare obiect are spațiul lui de adrese

Arrays

```
public class Main {
  public static void main(String[] args) {
    Pair[] pairs = new Pair[20];
    for (int i = 0; i < pairs.length; i++) {
      pairs[i] = new Pair();
    }
  }
```

```
}
```

Best practices

- nume sugestive
- clase/metode cu un singur rol
- main - numai câteva instanțieri + apeluri de metode

Laboratorul 2 - Constructori și referințe

Constructori

- funcții apelate automat la crearea unui obiect
- putem introduce restricții de integritate

Exemplu

```
class Student {
  private String name;
  private Integer year;

  public Student(String newName, Integer newYear) {
    name = newName;
    year = newYear;
  }
}
```

Constructorii:

- au numele clasei
- nu returnează explicit nici un tip (nici măcar void)
- poate avea oricâți parametri

Exemplu crearea de instanțe

- Student student = new Student("of", 2);
- warning: instanță vs clasă not the same thing

Constructorul implicit

- creat automat de Java dacă nu am definit noi deja unul
- inițializează câmpurile la 0/null/false în funcție de tipul lor
- nu mai este vizibil dacă am definit alți constructori

Cuvântul cheie *this*

- instanța curentă a clasei

Diferențierea între câmpurile și parametrii cu același nume

```
class Student {
  private String name;
  private Integer year;

  public Student(String name, Integer year) {
    this.name = name;
```

```
    this.year = year;
  }
}
```

Pasarea ca argument a referinței către instanța curentă

```
class Student {
  String name;
  Integer year;

  public Student(String name, Integer year) {
    this.name = name;
    this.year = year;
  }

  public void goToSecretary() {
    Secretary secretary =  new Secretary();
    secretary.handleStudent(this);
  }
}

class Secretary {
  public void handleStudent(Student student) {
    System.out.println("Oh no, here we go again " + student.name);
  }
}
```

Reutilizarea celorlalți constructori

```
class Student {
  String name;
  Integer year;

  public Student(String name, Integer year) {
    this.name = name;
    this.year = year;
  }

  public Student() {
    this("incognito", 0);
  }
}
```

Copy constructor

```
class Student {
  String name;
  Integer year;

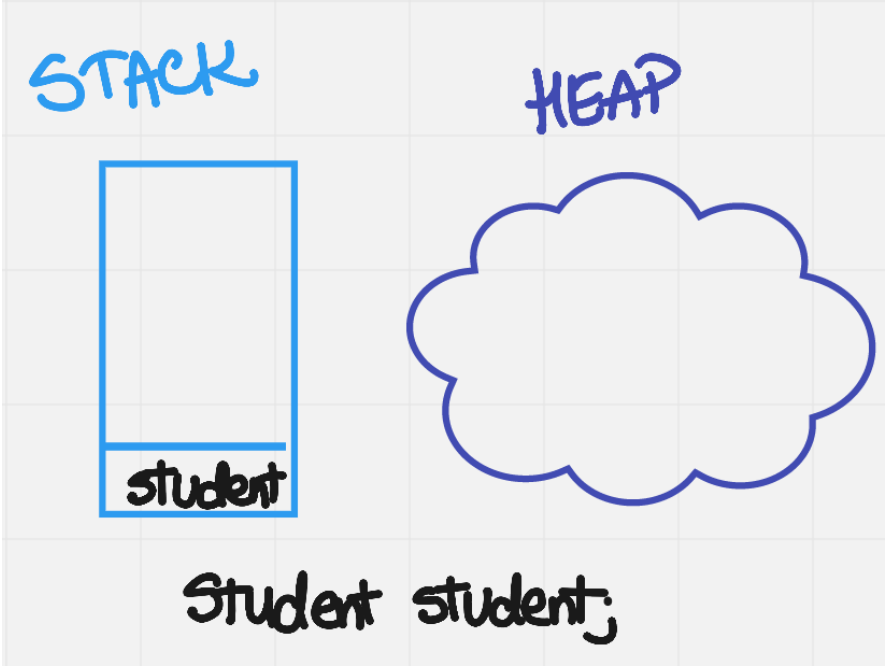
  public Student(String name, Integer year) {
```

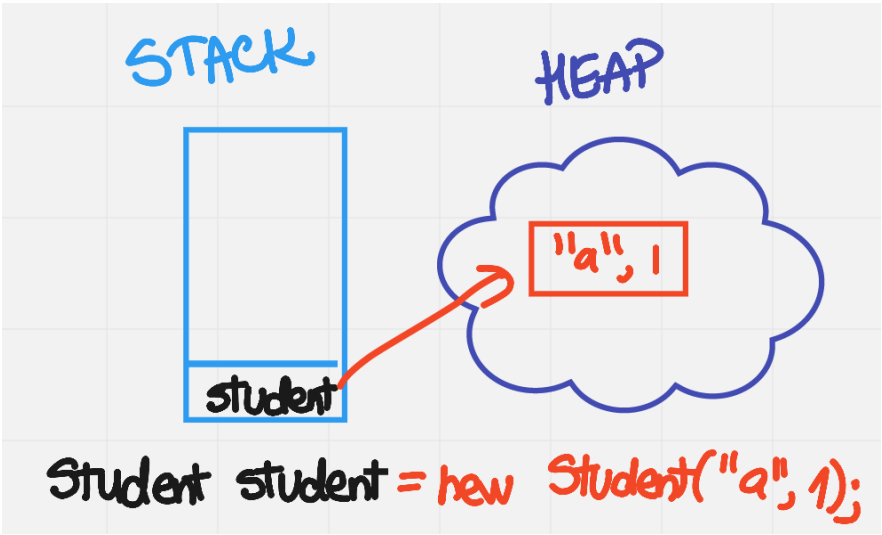
```
        this.name = name;
        this.year = year;
      }

      public Student(Student student) {
        this.name = student.name;
        this.year = student.year;
        /**
         * sau si mai scurt
         * this(student.name, student.year);
         */
      }
    }
```

Referințe

- nu avem pointeri, avem referințe
- obiecte alocate pe heap
- referințe pe stivă
- eliberarea automată a resurselor asigurată de Garbage Collection



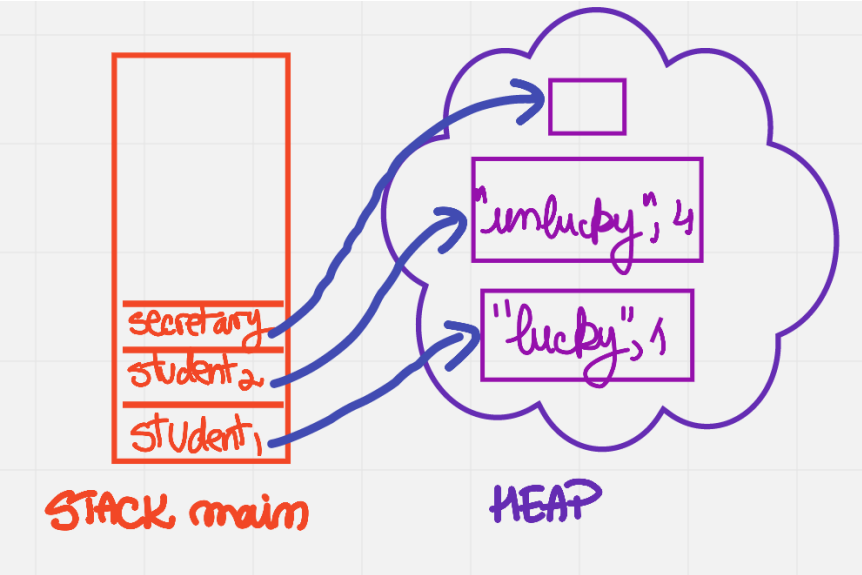


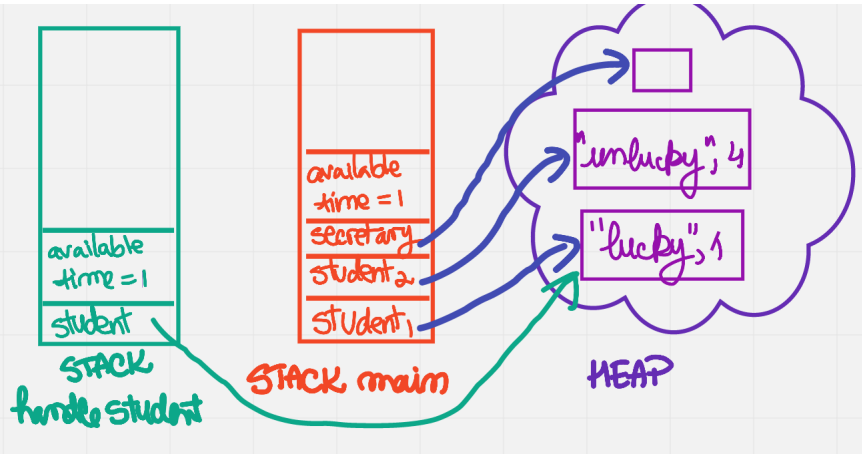

```
class Secretary {
  public void handleStudent(Student student, int availableTime) {
    System.out.println("Oh no, here we go again " + student.name);

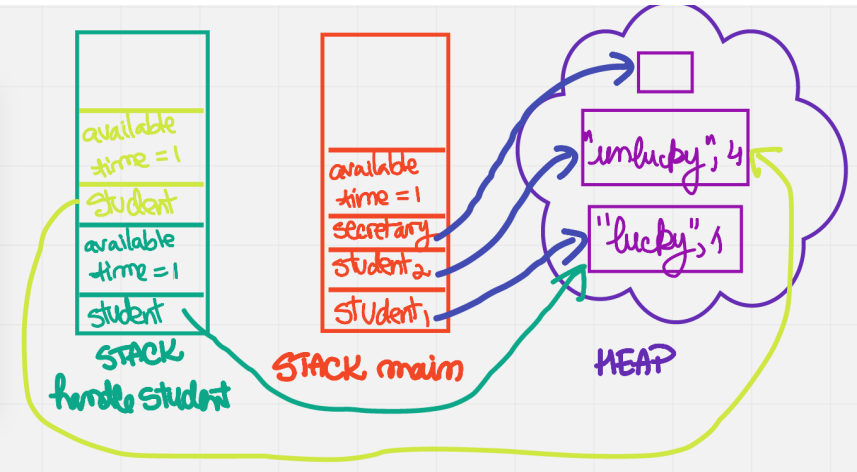
    if (availableTime > 0) {
      System.out.println("Okay, fine");
      student.isChecked = true;
      availableTime--;
    } else {
      System.out.println("Come back later, I don't have time now");
    }
  }
}
```

```
public class Main {
  public static void main(String[] args) {
    Student student1 = new Student("lucky", 1);
    Student student2 = new Student("unlucky", 4);
    Secretary secretary = new Secretary();
    int availableTime = 1;

    secretary.handleStudent(student1, availableTime);
    secretary.handleStudent(student2, availableTime);
    System.out.println(student2.isChecked);
  }
}
// afiseaza true
```







Transferul parametrilor

- tipurile primitive sunt copiate pe stiva => modificarile asupra lor nu sunt vizibile
- la instante se copiaza referintele => modificarile sunt vizibile

Metoda toString

- implementare predefinita in Java
- putem avea implementari proprii fiecarei clase

```
class Student {
  String name;
  Integer year;
  boolean isChecked = false;

  public Student(String name, Integer year) {
    this.name = name;
    this.year = year;
  }

  @Override
  public String toString() {
    return "Student with name " + name + " and year " + year + "\n";
  }
}
```

Laboratorul 3 - Agregare și moștenire

Agregare și compunere

- prezența unei referințe pentru un obiect într-o altă clasă

Agregare (aggregation)

- weak association
- obiectul container poate să existe fără obiectele agregate
- ex: portofel fără bani

Compunerea (composition)

- asociere puternică

- obiectul container NU poate să existe fără obiectele agregate
- obiectul container dispare atunci când dispar obiectele agregate
- ex: telefon fără procesor

```
class Professor {}

class Student {
  Professor tutor = null;

  public Student(Professor professor) {
    this.tutor = professor;
  }
}

class Course {
  Professor professor;

  public Course(Professor professor) {
    this.professor = professor;
  }
}
```

Inițializarea obiectelor conținute

- La definirea obiectului înainte de constructori
- În cadrul constructorului
- Chiar înainte de folosire (lazy initialization)

```
class Page {
    private String content;
    public int numberOfPages;

    public Page(String content, int numberOfPages) {
      this.content       = content;
      this.numberOfPages  = numberOfPages;
    }
}

class Book {
    private String title;                    // Compunere
    private Page[] pages;                          // Compunere
    private LibraryRow libraryRow = null;       // Agregare

    public Book(int size, String title, LibraryRow libraryRow) {
        this.libraryRow = libraryRow;
        this.title = title;

        pages = new Page[size];
```

```
            for (int i = 0; i < size; i++) {
               pages[i] = new Page("Page " + i, i);
            }
         }
      }

      class LibraryRow {
         private String rowName = null;                  // Agregare

         public LibraryRow(String rowName) {
            this.rowName = rowName;
         }
      }

      class Library {

         public static void main(String[] args) {
            LibraryRow row = new LibraryRow("a1");
            Book book = new Book(100, "title", row);

            // După ce nu mai există nici o referință la obiectul Carte,
            // Garbage Collector-ul va șterge (la un moment dat, nu
            // neapărat imediat) acea instanță, dar obiectul LibraryRow
            // transmis constructorului nu este afectat.

            book = null;
         }
      }
```

Moștenire (derivare)

- o clasă extinde o altă clasă
- preia funcționalitatea existentă și o modelează/adaugă alte funcționalități
- clasa extinsă = clasă părinte = clasă de bază = super clasă
- clasa care extinde = clasă copil = clasă derivată = sub clasă

Java NU permite moștenire multiplă

- orice clasă are un părinte unic
- se rezolvă problema rombului

Moștenire vs agregare

- moștenire - is a
- agregare - has a

Când folosim agregare și nu moștenire?

1. Clasa A extinde B
    - moștenire: A moștenește toate metodele clasei B
    - agregare: putem limita accesul numai la anumite metode

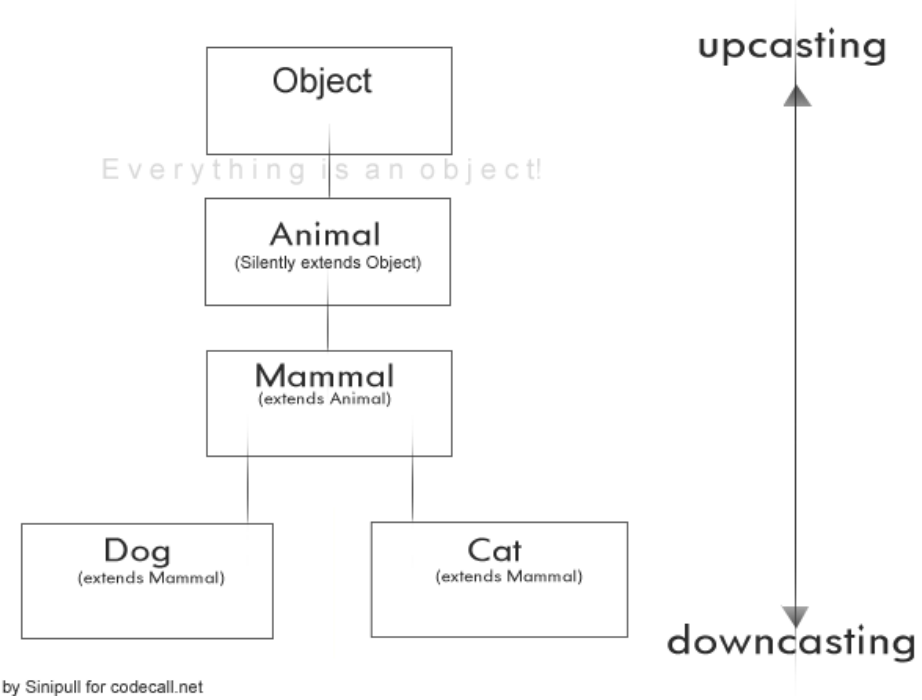
- e recomandat să folosim agregare dacă nu avem nevoie de toate metodele lui B

2. Vrem acces direct asupra obiectelor agregate
   - folosim agregare

Când folosim moștenire și nu agregare?

- când vrem să avem implementări mai specifice ale unei clase

Upcasting & downcasting



Upcasting

- cast de la clase specifice la clase mai generale
- se face automat
- ex: student.toString(), nu trebuie ((Object)student).toString()

Downcasting

- cast de la clase generale la clase mai specifice
- NU se face automat
- generează excepție dacă obiectul nu are tipul la care facem downcasting
- cum aflăm tipul unui obiect? operatorul instanceof

```
class Course {}

class POO extends Course {
  void joinLab() {
    System.out.println("POO lab");
  }
}

class AA extends Course {
  void joinSeminar() {
    System.out.println("AA seminar");
```

```
  }
}
```
Așa no
```
public class Main {
  public static void main(String[] args) {
    Course course1 = new POO();
    ((POO) course1).joinLab();

    Course course2 = new AA();
    ((AA) course2).joinSeminar();

    ((AA) course1).joinSeminar();
    ((POO) course2).joinLab();
  }
}
```
Așa da
```
public class Main {
  public static void main(String[] args) {
    Course course1 = new POO();
    if (course1 instanceof POO) {
      ((POO) course1).joinLab();
    }

    Course course2 = new AA();
    if (course2 instanceof AA) {
      ((AA) course2).joinSeminar();
    }
  }
}
```
Chiar și așa instanceof NU e deloc recomandat

- cod repetitiv și greu de citit
- cod greu de menținut
- nu respectă principiul OCP (open for extension, closed for modification)

Alternative

- Polimorfism
- Visitor

Polimorfism

- metode care depind de instanță (exact ce sub-clasă va avea la runtime)
- Suprascriere(overriding): funcție cu aceeași declarație, dar cu implementare diferită în copii vs părinte
- Supraîncărcare(overloading): funcție cu același nume, dar cu parametri diferiți

```
class Course {
  void join() {
    System.out.println("Join something");
```

```
  }

  void print() {
    System.out.println("Some course");
  }
}

class POO extends Course {
  void join() {
    System.out.println("JoinPOO lab");
  }
}

class AA extends Course {
  void join() {
    System.out.println("Join AA seminar");
  }
}
public class Main {
  public static void main(String[] args) {
    Course course = new POO();
    // Prints "Join POO lab", not "Join something"
    course.join();
  }
}
```

Exemplu suprascriere equals

```
class Course {
  String name;

  @Override
  public boolean equals(Object o) {
    if (this == o) {
      return true;
    }
    if (o == null || getClass() != o.getClass()) {
      return false;
    }
    Course course = (Course) o;
    return Objects.equals(name, course.name);
  }
}
```

Implicații moștenire

- metode/câmpuri protected - au acces și clasele derivate
- metode/câmpuri private - nu are acces decât super clasa
- constructorii nu se moștenesc (dar pot fi apelați cu super)

Super

- instanța clasei părinte
- folosit pentru a apela o metodă suprascrisă
- sau pentru a apela constructorul părinte

```
class Course {
  String name;

  public Course(String name) {
    this.name = name;
  }

  @Override
  public String toString() {
    return "a course with the name " + name;
  }
}

class POO extends Course {
  public POO(String name) {
    super(name);
  }

  @Override
  public String toString() {
    return "POO: " + super.toString();
  }
}
```

Laboratorul 4 - Static și final. Singleton Design Pattern

Cuvântul cheie final

- variabilele declarate cu final pot fi inițializate o singură dată
- daar obiectele către care pointează pot fi modificate intern

Obiecte immutable

- toate atributele unui obiect admit o unică inițializare
- nu putem schimba obiectul în sine (ex: informația pe care o stochează), ci doar referința către un alt obiect
- exemple: instanțele claselor String și Integer

Cum prelucrăm astfel de obiecte?

- prin instanțierea de noi obiecte și nu prin alterarea obiectelor înseși

```
public class Main {
  public static void main(String[] args) {
    String s1 = "POO";
    String s2 = s1.toUpperCase();
    System.out.println(s1 == s2);
    System.out.println(s1.equals(s2));
  }
}
```

String pool

- obiecte de tip String sunt immutable
- literalii de tip String sunt păstrați într-un pool
- limitează memoria utilizată prin inițializarea unui singur obiect către care pot pointa mai multe referințe

```
public class Main {
  public static void main(String[] args) {
    String s1 = "abc";
    String s2 = "abc";
    System.out.println(s1 == s2);
    System.out.println(s1.equals(s2));

    String s3 = new String("abc");
    String s4 = new String("abc");
    System.out.println(s1 == s2);
    System.out.println(s1.equals(s2));
  }
}
```

Probleme

```
public static String concatenationUsingTheStringClass() {
    String t = "Java";
    for (int i = 0; i < 10000; i++) {
      t = t + "POO";
    }
    return t;
  }
```

De ce e o problemă?

- nu ne pasă decât de rezultatul final
- umplem pool-ul cu string-uri intermediare pe care nu le mai folosim

StringBuilder

- avem un singur StringBuilder și nu câte un String pentru fiecare pas intermediar
- reduce overheadul

Soluție folosind StringBuilder

```
public static String concatenationUsingTheStringClass() {
    StringBuilder sb = new StringBuilder("Java");
```

```
    for (int i = 0; i < 10000; i++) {
      sb.append("POO");
    }
    return sb.toString();
  }
```

Alte utilizări ale cuvântului final

- aplicat unei clase - NU poate fi extinsă
- aplicat unei metode - NU poate fi suprascrisă

Cuvântul cheie static

- folosit pentru câmpuri/metode independente de instanțele clasei
- vor ocupa o locație unică în memorie, care nu depinde de obiectele create din clasa respectivă

```
class Student {
  private String name;
  private static int UNIVERSITY_CODE = 120;
  private static int STUDENT_COUNT = 0;

  public Student(String name) {
    this.name = name;
    STUDENT_COUNT++;
  }

  public static int getStudentCount() {
    return STUDENT_COUNT;
  }
}
```

Apelarea metodelor statice

- instance.staticMethod()
- ClassName.staticMethod()
- recomandată e a doua variantă pentru că e mult mai clar că metoda nu depinde de o instanță a clasei

Blocuri statice de cod

```
class Student {
  private String name;
  private static int UNIVERSITY_CODE;
  private static int STUDENT_COUNT;

  static {
    UNIVERSITY_CODE = 120;
    STUDENT_COUNT = 0;
  }

  public Student(String name) {
    this.name = name;
```

```
    STUDENT_COUNT++;
  }

  public static int getStudentCount() {
    return STUDENT_COUNT;
  }
}
```

Blocurile statice de cod

- apelate când clasa este încărcată
- încărcarea se face la prima referire în cod

Singleton Pattern

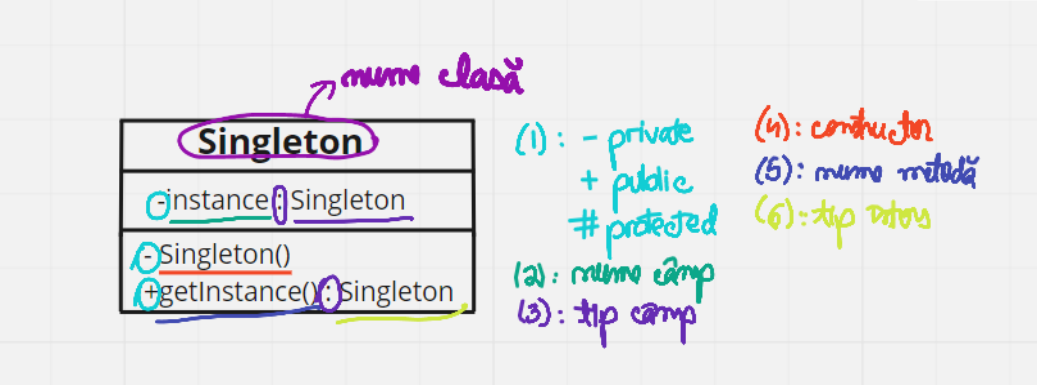
- restricționează numărul de instanțieri la un singur obiect

Utilizări

- în locul variabilelor globale
- obiecte de tip logger
- resurse partajate (ex: același fișier de input folosit de mai multe clase)
- în alte design pattern-uri (ex: Factory)

Warning: singleton e considerat antipattern

- introduce multe dependențe între clase
- îngreunează testarea individuală a claselor



Singleton lazy instantiation

```
class SingletonLazy {
  private static SingletonLazy instance = null;

  private SingletonLazy() {}

  public static SingletonLazy getInstance() {
    if (instance == null) {
      instance = new SingletonLazy();
    }
    return instance;
  }
}
```

Singleton eager instantiation

```
class SingletonEager {
  private static SingletonEager instance = new SingletonEager();

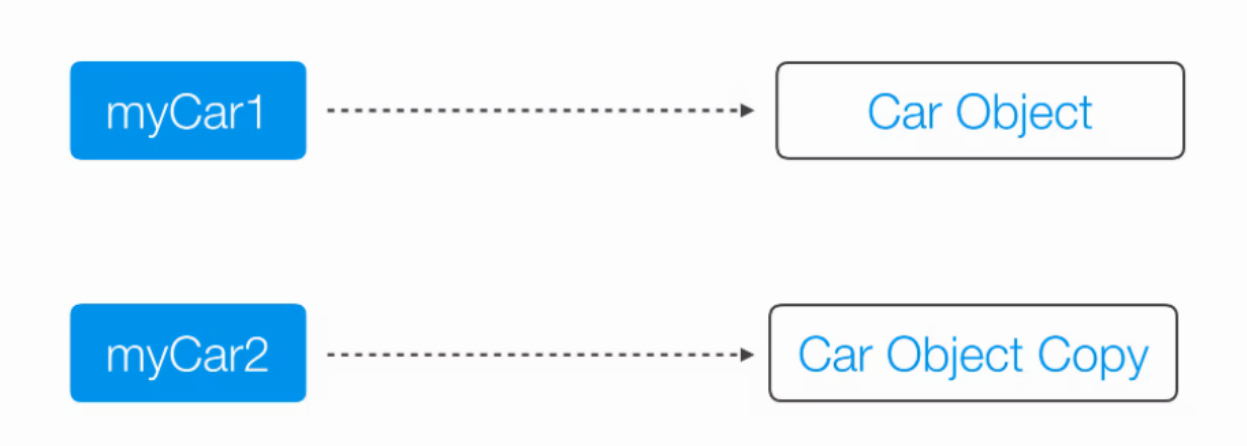
  private SingletonEager() {}

  public static SingletonEager getInstance() {
    return instance;
  }
}
```

Reference copy



Object copy



Shallow copy

```
class Account {
  private String username;
  private String password;
}

class Person {
```

```
  private Account account;

  public Person(Person firstPerson) {
    this.account = firstPerson.account;
  }
}
```

Deep copy

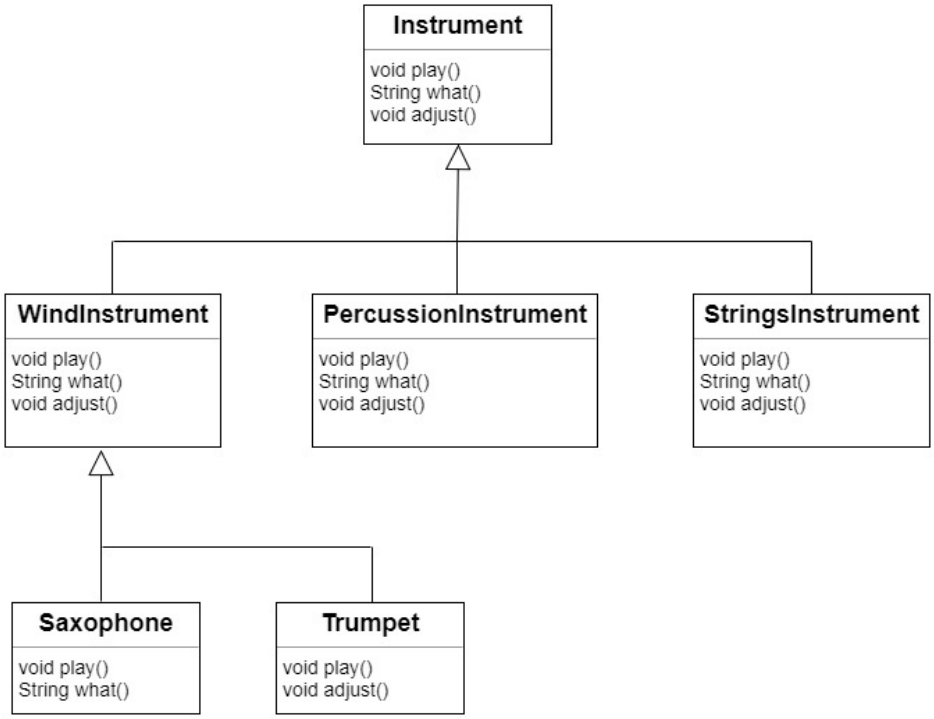
```
class Account {
  public String username;
  public String password;

  public Account(String username, String password) {
    this.username = username;
    this.password = password;
  }
}

class Person {
  private Account account;

  public Person(Person firstPerson) {
    this.account = new Account(firstPerson.account.username, firstPerson.account.password);
  }
}
```

Laboratorul 5 - Abstractizare



Principii POO de bază

- Abstractizare
- Încapsulare
- Moștenire
- Polimorfism

Abstractizare

- construirea unui șablon general pe care se bazează o anumită categorie de produse
- obiectele cunosc interfața, dar nu concret și implementarea

Clase abstracte

- vrem să implementăm doar o parte din metode
- clasa nu poate fi instanțiată

Interfețe

- avem doar o descriere a structurii, fără implementări
- o clasă poate extinde mai multe interfețe, dar nu poate moșteni mai multe clase

Clase abstracte

- ce au în comun toate clasele derivate?
- ce funcționalitate diferă = metode abstracte

Când folosim clase abstracte?

- vrem o interfață comună pentru un set de clase
- reutilizăm o serie de metode și membrii din această clasă în clasele derivate

Dynamic binding
- compilatorul determină implementarea potrivită la run time
- la baza polimorfismului

Putem instanția clase abstracte?
- Nope, eroare de compilare :(

Metode abstracte
- abstract return_type function_name(parameters);
- o clasă cu cel puțin o metodă abstractă = clasă abstractă

Moștenirea claselor abstracte
- dacă nu implementăm toate metodele abstracte => o altă clasă abstractă

Putem să marcăm o clasă abstractă fără a avea metode abstracte?
- Yess, este util dacă vrem să nu permitățm instanțierea unei clase

Exemplu clasa abstracta

```
abstract class Human {
  String name;

  abstract void getJob();

  public void callMe() {
    System.out.println("hello " + name);
  }
}

class Student extends Human {
  @Override
  void getJob() {
    System.out.println("Internship");
  }
}
```

Interfețe
- nume de metode, listă de argumente, valori întoarse daar fără implementare
- pot conține câmpuri care by default sunt final & static & public
- toate metodele declarate sunt public implicit

Crearea unei interfețe
- cuvântul cheie interface în loc de class
- interfață publică => trebuie să aibă același nume cu fișierul
- fără specificator de acces => package-private

Moștenirea unei interfețe
- de către o clasă: implements
- de către o interfață: extends

Metodele sunt package-private by default
- trebuie marcate ca public dacă vrem să fie așa

Exemplu interfețe

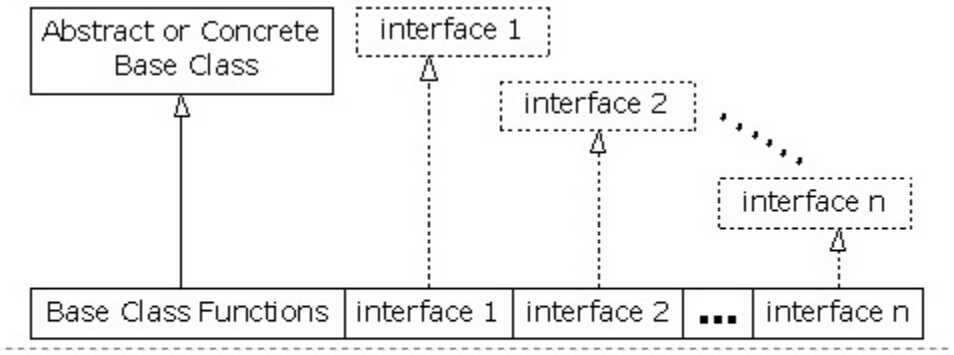
```
interface Human {
```

```
  boolean isAlive = true;
}

interface Student extends Human {
  public void cry();
}

class StudentLaPoli implements Student {
  public void cry() {
    System.out.println("crying in deadlines");
  }
}
```

Moștenire multiplă în Java




```
interface CanFight {
    void fight();
}

interface CanSwim {
    void swim();
}

interface CanFly {
    void fly();
}

class ActionCharacter {
    public void fight() {
    }
}

class Hero extends ActionCharacter implements CanFight, CanSwim, CanFly {
```

```
    public void swim() {
    }

    public void fly() {
    }
}

public class Adventure {

    static void t(CanFight x) {
        x.fight();
    }

    static void u(CanSwim x) {
        x.swim();
    }

    static void v(CanFly x) {
        x.fly();
    }

    static void w(ActionCharacter x) {
        x.fight();
    }

    public static void main(final String[] args) {
        Hero hero = new Hero();

        t(hero); // Treat it as a CanFight

        u(hero); // Treat it as a CanSwim

        v(hero); // Treat it as a CanFly

        w(hero); // Treat it as an ActionCharacter
    }
}
```

Metode default & statice în interfețe

Exemplu metodă default

```
interface Test {
  default void print() {
    System.out.println("default print");
```

```
  }
}

class CustomTest implements Test {
  void test() {
    print();
  }
}
```

Exemplu metodă static

```
interface Test {
  static void print() {
    System.out.println("default print");
  }
}

class CustomTest implements Test {
  void test() {
    Test.print();
  }
}
```

Laboratorul 6 - Clase interne

Ce sunt clasele interne?

- clase declarate în interiorul altei clase

Tipuri de clase interne

- clase interne normale (regular inner classes)
- clase anonime (anonymous inner classes)
- clase interne statice (static nested classes)
- clase interne metodelor (method-local inner classes) sau blocurilor

O clasă internă

- se comportă ca un membru al clasei în care a fost declarată
- are acces la toți membrii clasei în care a fost declarată, inclusiv cei private
- poate fi public, final, abstract, private, protected, static

Clase interne ”normale”

- definite în interiorul unei clase
- pot fi accesate doar la runtime printr-o instanță a clasei externe
- compilatorul creează fișiere .class separate pentru fiecare clasă internă (dar care nu pot fi executate separat)
- în interiorul clasei interne putem accesa referința la clasa externă folosind this (<nume_clasa_parinte>.this)

```
interface Engine {
   public int getFuelCapacity();
}
```

```
class Car {
   class OttoEngine implements Engine {
      private int fuelCapacity;

      public OttoEngine(int fuelCapacity) {
         this.fuelCapacity = fuelCapacity;
      }

      public int getFuelCapacity() {
         return fuelCapacity;
      }
   }

   public OttoEngine getEngine() {
      OttoEngine engine = new OttoEngine(11);
      return engine;
   }
}

public class Test {
   public static void main(String[] args) {
      Car car = new Car();

      Car.OttoEngine firstEngine = car.getEngine();
      Car.OttoEngine secondEngine = car.new OttoEngine(10);

      System.out.println(firstEngine.getFuelCapacity());
      System.out.println(secondEngine.getFuelCapacity());
   }
}
// afiseaza 11\n10\n
```

Observații

- constructorii devin privați
- putem crea instanțe de Car.OttoEngine prin intermediul interfeței Engine și a metodei getEngine
- folosim upcasting pentru a instanția clasa

```
interface Engine {
  public int getFuelCapacity();
}

class Car {
  private class OttoEngine implements Engine {
```

```
    private int fuelCapacity;

    public OttoEngine(int fuelCapacity) {
      this.fuelCapacity = fuelCapacity;
    }

    public int getFuelCapacity() {
      return fuelCapacity;
    }
  }

  public OttoEngine getEngine() {
    OttoEngine engine = new OttoEngine(11);
    return engine;
  }
}

public class Test {
  public static void main(String[] args) {
    Car car = new Car();
    Engine engine = car.getEngine();

    System.out.println(engine.getFuelCapacity());
  }
}
```

Cum putem crea instanțe cu alte valori pentru fuelCapacity?

- Parametru pentru metoda getEngine()

Clase anonime

- utilizări reduse ale clasei => nu are sens să definesc o nouă clasă
- instanțe ale unei clase moștenite (sau a unei interfețe extinse)
- suprascriu anumite metode

```
interface Engine {
  public int getFuelCapacity();
}

class Car {
  public Engine getEngine() {
    return new Engine() {
      private int fuelCapacity;

      public int getFuelCapacity() {
        return fuelCapacity;
      }
    };
```

```
  }
}

public class Test {
  public static void main(String[] args) {
    Car car = new Car();
    Engine engine = car.getEngine();

    System.out.println(engine.getFuelCapacity());
  }
}
```

return new Engine() {...}

- creează o clază care implementează interfață Engine, conform următoarei implementări + instanțierea clasei folosind new

Observații

- reduce volumul codului + crește lizibilitatea acestuia
- obiectul este instanțiat imediat după return folosind new
- numele clasei instanțiate este absent (știm numai supertipul)

Limitări

- nu au nume => nu știm cum să numim constructorii => nu vor avea constructori (diferiți de cel default)
- o clasă anonimă poate să implementeze o singură interfață (nu mai multe) sau să extindă o singură clasă, dar nu ambele în același timp
- nu pot folosi variabile declarate în metoda respectivă și nici parametrii metodei dacă nu sunt declarate cu final

Accesul claselor interne la variabilele declarate în metodă

- numai dacă sunt finale sau effectively final
- effectively final: nu o declarăm cu final dar odată instanțiată, nu îi modificăm valoarea

Expresii Lambda

- funcții anonime

Exemplu expresie lambda

```
interface Engine {
  public int getFuelCapacity();
}

class Car {
  public Engine getEngine() {
    return () -> 11;
  }
}

public class Test {
  public static void main(String[] args) {
```

```
    Car car = new Car();
    Engine engine = car.getEngine();

    System.out.println(engine.getFuelCapacity());
  }
}
```

Alte exemple expresii lambda

```
public class Test {
  public static void main(String[] args) {
    ArrayList<Integer> values = new ArrayList<>(Arrays.asList(1, 2, 3, 4, 5));
    values.removeIf((v) -> v % 2 == 0);
    values.forEach((v) -> System.out.println(v));
    values.forEach(System.out::println);
  }
}
```

Observații

- :: folosit pentru referențierea metodelor
- foarte multe funcții built in (spoiler PP sem 2): map, forEach, filter, reduce

Cum arată o lambda expresie care calculează suma a două numere a și b?

- (a, b) -> a + b

Clase interne statice

- nu avem nevoie de un obiect al clasei externe pentru a crea un obiect al clasei interne
- nu putem accesa câmpuri nestatice ale clasei externe din clasa internă

```
interface Station {
  public String getName();
}

class Car {
  static class GasStation implements Station {
    String name;

    public GasStation(String name) {
      this.name = name;
    }

    @Override
    public String getName() {
      return name;
    }
  }
}
```

Clase interne în metode

```
interface Engine {
  public int getFuelCapacity();
}

class Car {
  public Engine getEngine() {
    class OttoEngine implements Engine {
      private int fuelCapacity = 11;

      public int getFuelCapacity() {
        return fuelCapacity;
      }
    }
    return new OttoEngine();
  }
}
```

Clase interne în metode

- clasele sunt vizibile numai în interiorul funcției
- singurii modificatori care pot fi aplicați claselor: abstract, final

Clase interne în blocuri

```
interface Engine {
  public int getFuelCapacity();
}

class Car {
  public Engine getEngine(int fuelCapacity) {
    if (fuelCapacity == 11) {
      class OttoEngine implements Engine {
        private int fuelCapacity = 11;

        public int getFuelCapacity() {
          return fuelCapacity;
        }
      }
      return new OttoEngine();
    }
    return null;
  }
}
```

Explicații

- chiar dacă o declarăm numai în if, clasa se creează oricum
- why? numai la runtime o să știm pe care caz mergem

- clasa e vizibilă numai în blocul de if, nu și în rest (chiar dacă este creată)

Moștenirea claselor interne

- constructorul clasei trebuie să se atașeze de un obiect al clasei externe
- mai complicată pentru că trebuie să inițializăm și legătura cu clasa exterioară

```
class Car {
  class Engine {
    public void getFuelCapacity() {
      System.out.println("I'm a generic engine");
    }
  }
}

class OttoEngine extends Car.Engine {
  OttoEngine(Car car) {
    car.super();
  }
}

class Test {
  public static void main(String[] args) {
    Car car = new Car();
    OttoEngine ottoEngine = new OttoEngine(car);
    ottoEngine.getFuelCapacity();
  }
}
```

Utilizarea claselor interne

- clasa nu mai are utilitate în alte zone ale programului sau nu vrem să fie accesibilă din exterior
- ascunderea implementării unei interfețe
- nu putem extinde mai multe clase, dar putem defini clase interne cu acces la obiectul clasei exterioare

Laboratorul 7 - Overriding, overloading & Visitor pattern

Overloading

- mai multe metode cu acelasi nume dar cu semnaturi diferite
- semnatura = nume metoda + numarul si tipul parametrilor
- + exceptii

Tipul de return nu face parte din semnatura unei functii

Overloading ok

- modific fie tipul parametrilor, fie numarul lor

Supraincarcare = polimorfism static

- static = se face la compilare
- compilatorul alege metoda pe baza tipului referinterelor (la runtime, stim deja ce metoda apelam)

Overriding

- aceleasi metode (aceeasi semnatura + acelasi tip de return) implementate diferit la copii

Overriding = polimorfism dinamic

- alegem metoda la runtime
- numai la runtime o sa stim exact ce tip are o instanta

Polimorfismul dinamic

- abilitatea unei clase să se comporte ca o altă clasă de pe lanțul de moștenire

Reguli suprascriere

- aceeasi semnatura
- acelasi tip de return sau un tip care mosteneste tipul initial
- specificatorul de acces nu poate fi mai restrictiv
- metodele de static si final nu pot fi suprascrise
- constructorii nu pot fi suprascrisi
- nu poate arunca mai multe exceptii sau exceptii mai generale

Exemplu overriding

```
class Human {
  public void solve(String problem) {
    System.out.println("of");
  }
}

class Student extends Human {
  public void solve(String problem) {
    System.out.println("of Student");
  }
}
```

Adnotarea @Override

- complet optionala
- daar ajuta: cod mai usor de citit + compilatorul verifica tipul de return si parametrii functiei

int != Integer

- void f(int x) vs void f(Integer x)
- Integer permite si null, int nu
- (int = tip primitiv, Integer = wrapper)

Super

- constructorul din clasa parinte: super(....)
- metoda X din clasa parinte: super.x(...)
-

Laboratorul 8

Laboratorul 9

Laboratorul 10

Laboratorul 11

Laboratorul 12

Laboratorul 13